ConnectBox Software Specification

Version 2 · Oct 29, 2026 · @Kirk Wilson (replaces Version 1 of Oct 7, 2026)

ConnectBox software turns a small single-board computer into an offline Wi-Fi library: it runs its own hotspot and captive portal, indexes a USB drive into multilingual menus, and serves video, audio, documents and offline websites to any phone browser with no internet. This specification describes what the code in connectbox-pi and its sibling repositories does, as of the current master branch.

1. Scope, users and constraints

A ConnectBox serves one place at a time (a clinic, school, church or village) where there is no reliable internet. People join its Wi-Fi with the phone they already own and browse a media library in their own language.

Role	What they do
Visitor	Joins the Wi-Fi (default name "Connectbox - Free Media"), is sent to the menu by the captive portal, watches, reads, downloads, chats
Content preparer	Copies media onto a USB drive in language folders; the box builds the menus from it
Administrator	Changes Wi-Fi name, password, branding and settings in the admin pages; reads usage statistics
Organization	Optionally runs a central server that collects statistics and sends content packages to many boxes

Hardware: NanoPi NEO (main test unit), Raspberry Pi 3/4 and CM4, Orange Pi Zero 2, Radxa CM3. The model is read from `/sys/firmware/devicetree/base/model` at provisioning time. Boards run Armbian or Raspberry Pi OS (Debian Buster/Bullseye/Bookworm family).

Design constraints:

- Works with no internet at all; internet, when present, is used only for optional extras (online translation lookups for languages that do not ship with the box, central server sync). Interface translations for the 50 most spoken languages ship with the software.
- Must work on very old phones (iOS 9, Android 4–5 era browsers): plain pages, no app install.
- A ConnectBox can be dangerous to carry in some countries, so nothing stays logged in as administrator: the admin login ends when the admin page is left or after 5 minutes without use.
- Small boards: 512 MB RAM on the NEO, so conversions stream and only Python-standard or pure-Python libraries are used on the box.

- The Wi-Fi MAC address changes on every boot by design, so the box's address on an upstream network is never fixed. The wired (Ethernet) MAC is the opposite: fixed per box, set at boot from the processor's unique ID on the NanoPi NEO, because it is the Box ID the central server knows the box by.
- One USB drive holds all content; FAT32, exFAT, NTFS and Linux file systems (ext4 and others) are supported.

2. Repositories and components

connectbox-pi is the integration point: its Ansible playbook installs everything else onto the board. All repositories are on GitHub under the ConnectBox organization (Kirk's fork mirrors connectbox-pi).

Repository	Runs on	What it provides
connectbox-pi	Box	Ansible playbook and roles; `mmiLoader.py` (USB indexer); `PxUSBm.py` (USB mounter and network recovery); nginx sites; scripts that patch the compiled app; translations; tests
connectbox-mediainterface	Box (browser)	The Angular/Ionic media app visitors use; its GitHub Actions publish a `latest` release on every push to `main`. Also ships `lazyLoader.py` for OpenWell packages
connectbox-admin-ui	Box (browser)	The admin pages at `/admin`
connectbox-manage	Box	Node.js backend on port 5002 behind the admin pages and the `connectboxmanage` command
connectbox-client	Box	Classic (non-enhanced) web interface, cloned by the webserver-content role
simple-offline-captive-portal	Box	Flask captive portal on port 5001
connectbox-wifi-configurator	Box	Helper that writes hostapd Wi-Fi settings
connectbox-hat-service	Box	OLED display, buttons, battery level and safe shutdown
connectbox-chathost	Server	Central server: statistics, chat

Repository	Runs on	What it provides
		relay, OpenWell package links, MediaBuilder (Bolt CMS)
connectbox-tools	Build Pi	Scripts that provision a board and turn it into a release image
connectbox-reports, connectbox-access-log-analyzer	Server or PC	Usage reporting from logs
armbian-build	Build machine	Board operating system images

3. Provisioning

A box is built by running one Ansible playbook against a freshly flashed board: `cd ansible && ansible-playbook -i <device_ip>, site.yml -e wireless_country_code=US`. `site.yml` loads variables for the OS family, distribution and board, then runs the `connectbox-pi` role, whose dependencies install the system in this order:

Order	Role	What it installs or changes
1	bootstrap	Core files: mmiLoader.py, PxUSBm.py, ConnectBoxManage.sh, shutdown scripts, brand.j2, translations (46 languages), logrotate; Python libraries (mammoth, openpyxl, xlrd, olefile) and antiword; on Armbian holds the kernel, the image's armbian-* packages and base-files so the final upgrade cannot replace them; on the NEO enables i2c0 and all three USB host ports (usbhost0-2)
2	hat-service	Larger /tmp and swap; the OLED, buttons and battery service (neo-battery-shutdown, pip-installed from connectbox-hat-service on GitHub, run unbuffered so its output reaches the journal)
3	dns-dhcp	dnsmasq (DHCP + DNS that

Order	Role	What it installs or changes
		answers every name with the box) and avahi
4	network-interfaces	Removes NetworkManager and systemd-resolved; phone tethering tools; interface files
5	wifi-ap	hostapd access point and the Wi-Fi configurator
6	mikegleasonjr.firewall	iptables: allow SSH, DHCP, DNS, mDNS, HTTP and HTTPS in
7	enhanced-content	Downloads the latest media app and admin UI releases, ffmpeg, icons, types.json additions, and runs the patch_*.py scripts on the app and the admin pages
8	nginx	Five sites (see section 4), home button snippet, gzip, caching rules
9	kiwix	kiwix-tools 3.8.2 for the board's CPU, `kiwix-serve` service, empty library
10	captive-portal	Flask portal (pip-installed from simple-offline-captive-portal on GitHub) in its own virtualenv, systemd unit
11	webserver-content	Classic interface; Flask + gunicorn chat/admin API on port 5000
12	usb-content	systemd-udevd settings; removes the retired udev automount rule
13	sample-content, rpi-permissions, wlan-set, remove-lz4	Mount point, permissions, final Wi-Fi settings, cleanup
last	image-preparation (optional)	Clears per-box state (admin session key, logs, Wi-Fi regulatory domain), resets the

Order	Role	What it installs or changes
		first-boot partition expansion flag and removes Armbian build leftovers (/armbian.key, qemu core dumps) before an image is made

The hardware-only roles are skipped for an AWS build (`aws_instance`). Device settings live as JSON in `/usr/local/connectbox/brand.j2` (the single source; `brand.txt` was retired), read and written with `jq` by `ConnectBoxManage.sh` and directly by connectbox-manage.

4. Network and web serving

The box is its own small network: hostapd runs the access point, dnsmasq hands out addresses on `10.129.0.1` and answers every DNS name with the box, and nginx on port 80 sends each request to the right service. Only nginx, SSH, DHCP, DNS and mDNS are reachable from outside; every backend listens on 127.0.0.1.

Captive portal. Phones probe a known address when they join Wi-Fi (`/generate_204`, `/hotspot-detect.html`, `/ncsi.txt`, `/connecttest.txt`, `/.well-known/captive-portal` and others). nginx's default site passes these to the Flask portal (port 5001), which answers each phone family in the way that makes it open a sign-in window showing the ConnectBox menu. The portal keeps per-client session timing so a rejoin is recognised, and keeps special handling for old iOS and Android versions on purpose. Typing `gowifi.org`, `wi.fi` or `go` also reaches it.

The portal page ("Connected to … Wifi", a link to the menu and, on Android 10 and later, a hint to choose "Use this network as is") is shown in the first of the phone's Accept-Language languages that the box has content in, otherwise in the box's default language (English when the content has an en or eng folder, otherwise the first language folder), otherwise English. Its text comes from the same translation files as the menu (PORTAL_* labels, section 8); right-to-left languages are laid out right to left.

Upstream connection (optional). A second Wi-Fi interface (client mode), Ethernet, or a tethered phone over USB can give the box internet. dhcpcd must not manage the client Wi-Fi interface (`denyinterfaces` placed before any `interface` block), otherwise the box takes two addresses. PxUSBm restarts hostapd and checks interfaces if the access point drops. Use one or two Wi-Fi adapters: PxUSBm puts the access point on the adapter that can run one (a Realtek RTL88xx first) and the client on the other; with three or more it changes nothing, so the access point may end up on an adapter that cannot run one.

Port	Service	Reached at
80	nginx (enhanced site for `connectbox`, `.local`; captive-portal default site; classic, icon-only and static sites when chosen)	Everything below

Port	Service	Reached at
5001	Captive portal (Flask + gunicorn)	Captive-portal probes
5002	connectbox-manage (Node.js 22, PM2)	`/admin/api`, `/chat`
5000	Flask chat and admin API (`python/main.py`, gunicorn)	Classic interface
8090	kiwix-serve	`/kiwix/` (ZIM files)

The enhanced site (`connectbox_enhanced.conf.j2`) serves the media app from `/var/www/enhanced/content/www`, the admin UI at `/admin`, USB files at `/content/` and adds these behaviours:

- `/` and `/index.html` are rewritten to `/tmp/connectbox-indexing.html` while a full index runs, and are always sent with `Cache-Control: no-cache`.
- A floating home button is inserted before `</body>` of every web-content and ZIM page (`subs_filter`, needs `libnginx-mod-http-subs-filter`).
- A 404 under `/assets/content/<lang>/html/` returns to the start page of the item the link was clicked in (from the Referer), not a blank page.
- `/kiwix/` is proxied with `^~` so the `.json` rule never takes ZIM files; Kiwix HTML is sent `no-cache` because nginx changes it.
- `/build/` and `/admin` are sent `no-cache` so app updates reach browsers; text assets are gzipped.

5. USB content pipeline

Plugging in a USB drive is the normal way to load content: PxUSBm mounts it and starts mmiLoader, which either restores the saved menus in seconds or indexes the whole drive (minutes on a large drive). Removing the drive clears the menus.

1. **Mount (PxUSBm.py).** A systemd service polls `lsblk` about every 3 seconds. A new drive is mounted at /media/usb0: its first partition (sda1) or a whole disk with a file system and no partition table. Before mounting, a FAT drive is checked with dosfsck -a when it is marked as not cleanly removed or as having had a disk error (a drive taken out of a ConnectBox always is, since it cannot be ejected); an NTFS drive gets ntfsfix -d; other file systems are not checked. A check is never run on a mounted drive, is stopped after 2 minutes (the drive is then mounted anyway) and shows "Checking USB" on the front panel meanwhile. FAT and NTFS drives get the utf8 option; NTFS is read through ntfs-3g. If a drive is no longer mounted where PxUSBm put it (the front panel's copy commands move it), PxUSBm forgets that slot so the drive comes back at /media/usb0. PxUSBm is the only mounter: a udev mounter was retired because the two raced.
2. **Start the loader.** On its next poll PxUSBm starts mmiLoader.py in the transient systemd unit `connectbox-loader`; the marker `/tmp/.usb0_indexed` stops a second start. mmiLoader itself refuses to run twice: the marker `/tmp/creating_menus.txt` means another run is in

progress, unless no other mmiLoader process is running, in which case it is a leftover and is removed.

3. **Prepare.** On Linux file systems mmiLoader adds read permission for everyone where it is missing (like `chmod -R a+rX`, symlinks skipped) so nginx can read files written by another computer's user. It rebuilds the Kiwix library from the drive's `.zim` files, then removes the old menus.
4. **Fast path: `saved.zip`.** If `content/saved.zip` (or one at the USB root) exists, it is extracted into `/var/www/enhanced/content/www/assets/content/`, translations and right-to-left flags are refreshed, and the run ends.
5. **Full index.** Otherwise mmiLoader writes a "Loading new content" page (`/tmp/connectbox-indexing.html`, progress "N / M", refreshing itself; it holds the message in every language on the drive and shows the one matching the phone's language, else the default), walks each language folder, builds cards and JSON (section 6), extracts thumbnails with ffmpeg, removes the loading page and writes saved.zip last. The front panel shows "Indexing 45% / of 778 files". It checks the drive is still mounted before each folder and before writing saved.zip, so a pulled drive never leaves a broken cache.
6. **Remove.** When the drive disappears PxUSBm stops the loader, empties the Kiwix library, runs `mmiLoader.py --clear` (empty English menu, default languages) and lazily unmounts. If the box has content of its own, it is indexed again (next paragraph).

Deleting `saved.zip` from the drive forces a full index on the next insert; the admin's "Load Content From USB Flash" does the same. Thumbnails are cached on the drive as hidden `.thumbnail-<lang>-<slug>.png` files, so re-indexing is faster than the first run.

The box's own content. The front panel's "copy from USB" copies a drive's content/ to `/media/usb0/content` on the SD card. A drive plugged in is mounted on top of /media/usb0 and shown instead. Whenever nothing is mounted at /media/usb0 and the box has its own content, PxUSBm starts the loader once for it (marker `/tmp/.internal_indexed`, cleared when a drive is pulled out; 20 seconds after start-up, so a drive that is plugged in is mounted first). mmiLoader then carries on while `/media/usb0/content` exists instead of requiring a mounted drive.

6. Content model

mmiLoader turns folders and files into cards, each described by JSON the media app reads: per language, `<lang>/data/main.json` (the card list), one data file per card, `types.json` (extension map), `interface.json` (menu wording) and a shared `languages.json`. Media are linked into a flat `<lang>/media/` folder; converted pages go to `<lang>/html/<slug>/`. The full layout rules are in the USB Content Structure Reference.

Folder types (decided per folder, in priority order):

Folder	Rule	Becomes
Web content	Has `index.html` / `index.htm`	One web card opening that site; optionally also a downloadable zip
Android app	Has `AndroidManifest.xml`	One app download card

Folder	Rule	Becomes
Complex	Has subfolders at any depth	A generated file-browser web page (`indexer.py`)
Collection	More than 2 files, no subfolders	One card with an episode list
Singular	1 or 2 files	One card per file
Language root	`en/`, `es/`, `zh-CN/` …	Not a card; its loose files are singular cards

Card pictures come from folder art (`folder.png`, `folder.jpg`, `cover.jpg`, `album.art.jpg`, `front.jpg`, then any picture), a video frame from ffmpeg (seeking past black frames), cover art inside audio files, the picture itself, or an icon for the file type. Hidden control files in `content/` change behaviour: `.language` (language of a drive with no language folders), `.compress` (zip of every multi-file folder), `.webcompress` (zip of each web content folder for download; off without it, because zipping folders of videos is slow and doubles the space they use).

File types. `types.json` maps about 55 extensions to a media type, MIME type and icon; files with other extensions, or names starting with `.` or `_`, are skipped. Videos must be H.264 in MP4 to play in browsers. Two files with the same name in different folders share a link only if they are identical copies; otherwise the later one is linked as `<name>--<folder><ext>` (`unique_media_name`), keeping slugs and statistics stable.

Office files shown as web pages. `make_document_web_page()` converts these at index time into `<lang>/html/<slug>/index.html`; the card becomes a web card but keeps the file's MIME type, so the app shows a details page with an open (book) button and a download of the original. A failed conversion leaves a download-only card.

Type	Library	What the page shows	Not kept
Word `.docx`	mammoth 1.13.0	Text, headings, lists, tables, pictures, links	Headers, footers, text boxes, columns
Excel `.xlsx` / `.xls`	openpyxl 3.1.5 (streaming) / xlrd 2.0.1	One table per visible sheet, saved formula results, first 2,000 rows and 100 columns, sheet links	Charts, colours, number formats
PowerPoint `.pptx`	Python standard library only	One numbered section per visible slide: title, bullets with levels, text boxes, pictures, tables, speaker notes	Slide design, animations, charts, diagrams
Old Word .doc	antiword (DocBook	Text, headings,	Pictures, manual line

Type	Library	What the page shows	Not kept
	output)	bold/italic/underline, lists (as bullets), tables	breaks; files under about 300 characters stay download-only
Old PowerPoint .ppt	olefile 0.47 + own record reader	Slide text with bullet levels, speaker notes, in slide order	Pictures; tables shown one line per cell; password-protected, PowerPoint 95 and damaged files stay download-only

A `.doc` or `.ppt` with a PDF of the same name in the same folder (any letter case) is not converted: the PDF is the one to read and the old file stays a download. Other formats stay download-only. Pages are full HTML with right-to-left direction for RTL languages.

ZIM files (offline websites) are served by kiwix-serve. Each `.zim` becomes its own web card, even inside a collection, with title, description and icon from the Kiwix library (unreadable or invisible icons replaced by a web icon, dark ones placed on white). Its page redirects to `/kiwix/content/<name>/`. A ZIM tagged with several languages is also listed in the box's other matching languages; TED ZIMs open in the menu's language. Kiwix's own search works inside each ZIM.

7. User interface

Visitors use the enhanced interface: the connectbox-mediainterface app (Angular/Ionic), downloaded as its `latest` GitHub release and then changed on the box by idempotent `patch_*.py` scripts and inline shell patches in the enhanced-content role. Changing the app this way keeps boxes working with the published release while ConnectBox-specific fixes are made.

What a visitor sees. A home page of cards grouped by category and "For You", a Language button, search, collections with episode lists, a details page per item with open and download buttons (the download button shows the file size), built-in players and PDF/EPUB readers, a chat page, and a Configuration link to the admin pages. Web content and ZIM pages carry a home button back to the menu.

Patch	Bundle	Behaviour
`goToDetails`	6.js	Web cards open straight away (same tab) and report the view to `/admin/api/weblog` themselves; Word, spreadsheet and presentation cards go to the details page like PDFs
`patch_zim_download.py`	3.js	Details page: ZIM cards have no download; Office cards download the original from

Patch	Bundle	Behaviour
		`media/<file>` and open with the book icon in the same tab
`patch_download_size.py`	0, 1, 3, 4, 5, 7.js	File size next to every download button, from a HEAD request; nothing shown if unknown
`loadData` language fix	3.js	On a language change, return to the root page first (prevents "media missing!")
`episode slug match`	3.js	A details page opens the episode that was tapped (matched by slug, not title), so a PDF next to a same-named .doc no longer opens the .doc
`patch_language_button.py`	6.js	Language button shows the active language's own name
`patch_footer_translation.py`	6.js	Configuration link in the menu's language
`patch_chat_rtl.py`	2.js, main.css	Translated chat labels, per-message text direction, bubble placement
`patch_saved_language.py`	main.js	A returning visitor's saved language is refreshed from the current `languages.json`
`patch_browser_language.py`	main.js	First-time visitors start in the first browser language the box has; a language picked in the menu always wins
`patch_admin_login.py`	admin.bundle.js, login.bundle.js	Configuration always asks for the password; leaving the admin page logs out; Exit goes to the main screen; keepalive once a minute; no default password on the login page

Each script exits non-zero when the text it expects is missing. Because every push to mediainterface `main` publishes a new release, all patches must be re-run against a new build before boxes take it.

The classic, icon-only and static-site interfaces remain in the playbook as alternatives (`enhanced_interface: False`).

8. Languages

Each top-level folder on the drive named with a language code (`en`, `es`, `ar`, `zh-CN`, or 3-letter codes such as `spa`) becomes a language in the menu; a drive with no language folders is English unless `content/.language` says otherwise. Codes are normalised with `languageCodes.json` (`per` → `fa`, `spa` → `es`), and the same mapping places multi-language ZIM files.

- **Menu wording.** The app only has English, so mmiLoader writes each language's `interface.json` from `/usr/local/connectbox/translations/<code>.json`. Files ship for 46 languages (the 50 most spoken; Chinese varieties use Simplified, Hong Kong or Taiwan Traditional Chinese, Egyptian Arabic has its own file), so a box needs no internet for them. Any other language is translated online once (MyMemory) when the box has internet and saved there; offline it stays English and is retried next time. Each entry stores the English it came from, so hand corrections stick until the English changes. The same strings serve pages mmiLoader writes itself (loading page, "Speaker notes") and the captive portal (PORTAL_* labels).
- **Right-to-left.** Arabic, Egyptian Arabic, Farsi, Hebrew, Urdu, Pashto, Sindhi, Western Punjabi, Kurdish (Sorani), Uyghur, Yiddish and Dhivehi get "rtl": true in `languages.json`; the app, converted pages, chat and the captive portal flip direction.
- **Phone languages.** `languages.json` lists each language's folder code first and then the codes phones report for it (for example arz: ar-EG; tl: fil; id: in). The app (first visit), the loading page and the captive portal start in the first phone language the box has, exact code first, then base language; a language picked with the Language button is saved and always wins in the app. The default language in languages.json is the English entry (folder en, eng or en-…) when there is one, otherwise the first language; it is also corrected when an older saved.zip is restored.
- **Language button** shows each language's own name from `languageCodes.json`.
- **Not translated:** content titles (file names), the brand name, chat messages.

9. Administration, statistics, chat and display

The admin pages (`/admin`, connectbox-admin-ui) talk to connectbox-manage on port 5002, which reads and writes `brand.j2` and runs system commands; the same calls are available on the box as `connectboxmanage get|set|do …` (which fails with a traceback if port 5002 is down).

- **Login:** password-protected, and never long-lived. The Configuration link always leads to the login page; the session ends when the admin page is left (Exit, Back, another link, closing the tab), and connectbox-manage refuses a session idle for 5 minutes (the page sends a keepalive once a minute while open). Exit goes to the main ConnectBox screen. The password is checked by openssl on its standard input, never through a shell, and the login page does not show the default password. The session cookie is signed with a random key created on the box on first start (`/usr/local/connectbox/admin_session.key`, root only), never a key in the source.

- **Settings:** admin password, Wi-Fi name (SSID), channel, optional WPA passphrase, country code, host name, branding (name, logo, intro text), upstream Wi-Fi client, central server address and authorisation.
- **Actions:** reload content from USB (deletes `saved.zip` and re-indexes), load or refresh OpenWell packages, sync with the server (`phonehome.py`), view logs, reboot, shut down, wipe.
- **Statistics:** the app reports each view or download with `PUT /admin/api/weblog`; the admin pages show the top items per hour, day, week, month and year and let them be downloaded or cleared. Statistics can be turned off (`disable_stats`).
- **Chat:** `PUT /chat` stores messages that every visitor sees; each message carries its sender's text direction.
- **OLED display and buttons (connectbox-hat-service, neo-battery-shutdown service):** status pages with battery level, connected users, IP address and the most-used files per period; "Checking USB" while a drive is checked before mounting; indexing progress ("Indexing 45%"); holding both buttons for 4 seconds opens the admin pages: copy from USB to the box, copy from the box to USB and erase, with progress ("Copying 45%" and the size), an error page that stays until a button is pressed, and one checkmark when the re-index has finished. Holding one button 4 seconds or more, with the other never pressed, shuts the box down safely (AXP209 power chip set over i2c, then shutdown -h). The panel removes a status file left by a loader that is no longer running.

10. Central server and OpenWell

A box works alone; an organization can also run a central server (connectbox-chathost) that many boxes report to when they get internet.

- **Sync.** When the box is online (tethered phone or upstream Wi-Fi) `phonehome.py` sends its usage statistics and chat messages to the server named in the admin pages and fetches what is waiting for it. It runs every 10 minutes from cron (as www-data) and stops at once while no server URL or site name is set. It identifies the box with “Bearer <Box ID>|<key>”: the Box ID is the wired MAC with dashes, the key is `server_authorization` in brand.j2, made by phonehome.py the first time it runs or set by the server. The build installs the maintained phonehome.py (it reads brand.j2); the old copy in scripts/ read brand.txt and failed.
- **OpenWell content packages.** The admin's Web Server section lists packages from the server (`get.subscriptions`); subscribing writes `assets/content/subscription.json`. Nothing downloads until the admin presses "Download Missing Content Files", which runs `lazyLoader.py` (from mediainterface) to fetch the package's menus and media. Packages are built on the server with MediaBuilder (Bolt CMS), linked at `/chathost/link/openwell`.
- **Server build.** The chathost playbook targets Ubuntu 24.04: Docker with compose v2, nginx with Let's Encrypt HTTPS, data under `/srv/connectbox`, secrets in an untracked `secrets.yml`, application and database ports bound to 127.0.0.1. It has been checked (syntax, nginx config, dependency install) but not yet run on a real server.

11. Security, reliability rules and invariants

These rules come from failures seen on real boxes; changes must keep them true.

Rule	Why
Backends listen on 127.0.0.1 only; the firewall admits SSH, DHCP, DNS, mDNS, HTTP, HTTPS	Visitors reach services only through nginx
Admin session key is per box and deleted when an image is prepared	The old fixed key was public and allowed forged admin logins
No long-lived admin login: the session ends when the admin page is left or after 5 idle minutes	A phone left logged in as administrator is dangerous in some countries
User input is never put into a shell command for the password (login, set password)	The old login ran the typed password as a root shell command, before login
PxUSBm is the only USB mounter and owns network recovery	A second mounter and a separate watchdog raced it
dosfsck/ntfsfix are never run on a mounted drive	Linux marks a mounted FAT drive dirty; dosfsck then "repaired" it while in use
`denyinterfaces` before any `interface` block in dhcpcd.conf; never reconfigure the client Wi-Fi over SSH on that link	Otherwise two addresses, or the box drops off the network
`saved.zip` is written last, after checking the drive is still mounted	A pulled drive must not leave a broken cache
The loading page is keyed on `/tmp/connectbox-indexing.html`, not creating_menus.txt	creating_menus.txt is the front panel's status text and single-run marker
Scripts use Unix line endings (`.gitattributes`)	A Windows line ending makes a script fail silently on the box
`brand.j2 must stay valid JSON and is always written to a temporary file and renamed into place`	Several programs parse it; the front panel re-reads it every pass and crashed on a half-written file
`mimeType` in card JSON is the real MIME type (`video/mp4`)	Browsers decide playback from it
Captive-portal checks for old iOS and Android stay	Deployed phones are that old
The Wi-Fi MAC (and so the upstream address) changes every boot	Intentional; do not pin it
The wired (Ethernet) MAC is fixed per box: set at boot from the processor ID (fixed_eth_mac.sh, udev rule)	It is the Box ID the central server knows the box by; the NEO kernel picked a random one every boot

Rule	Why
Node.js 22 lives in /usr/local; /usr/bin/node, npm, npx and pm2 link to it	PxUSBm runs /usr/bin/pm2; without the link it crash-looped and took both Wi-Fi networks down
brand.j2 values: whole numbers are stored as numbers, everything else as text	parseInt alone saved a key such as 405b2056-… as the number 405
One or two Wi-Fi adapters	With three or more, PxUSBm leaves the access point where it is, possibly on an adapter that cannot run one

Known risks: the captive portal pins old library versions (Flask 1.1.1, gunicorn 20.0.4) with open advisories, kept for old-phone compatibility until tested; the patch scripts depend on exact compiled text and must be re-checked for each app release.

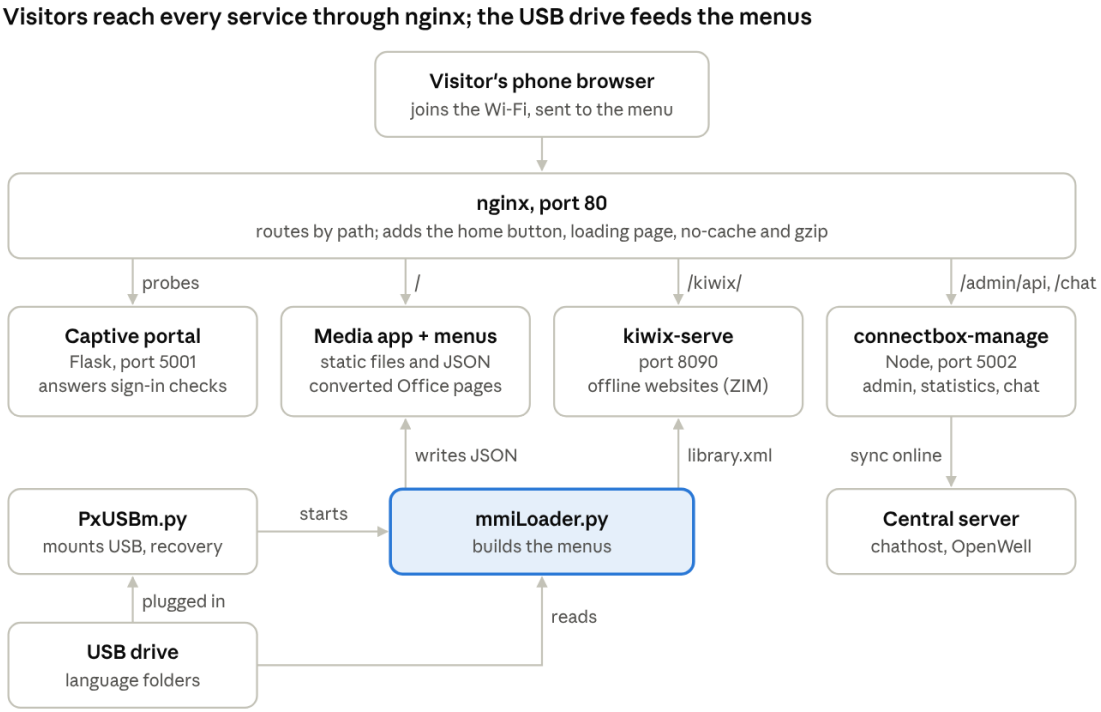
12. Testing, images and releases

Activity	How
mmiLoader tests	`ansible/roles/bootstrap/files/test_mmiLoader.py`, 316 checks in 22 scenarios (folder types, translations and phone codes, ZIM, Word, spreadsheets, presentations, old .doc/.ppt, icons, duplicate names, the box's own content, progress text; S15 has one out-of-date expectation); run after every mmiLoader change; kept Python 3.7-compatible
USB mount tests	`test_PxUSBm_mount.py` simulates `PxUSBm.mountCheck()` with fake `lsblk` output
Front panel tests	Simulation harness with fake buttons, display and mounts for the admin commands (copy, erase, progress, one checkmark, stale status file), kept in the Claude working folder (hatbox/tests35, tests38), not yet in a repository
On-device check	Copy the changed file to the test unit from git, not from the Windows working tree (or strip Windows line endings), force a re-index, check in a phone browser; patch scripts are run against the real release bundles

Activity	How
Lint	`ansible-lint ansible/site.yml`
Release image	connectbox-tools on the build Pi: `make_cb.py` provisions a board, `prepare_release.py` runs image-preparation and writes the image to `deployment/Images`; images are published on the connectbox-pi releases page (see `docs/making_an_image.md`)
App release	Every push to connectbox-mediainterface `main` builds and publishes `latest`, which new boxes download

Work happens on `master` in connectbox-pi with conventional commit messages, pushed to both the ConnectBox organization and Kirk's fork. Code comments follow the repository's policy: every function, loop and section explains what it does and why.

13. Architecture at a glance



ConnectBox components on the box and the central server

Phones only ever talk to nginx; the USB drive reaches them through PxUSBm and mmiLoader, which write the menus and the Kiwix library. The OLED display and buttons (hat service) run alongside and read the same statistics.